

I-SHOOT MO, I-SHOOT MO

COMPLETE CONSTRUCTION AND PROGRAMMING

MANUAL

Approved IoT Carnival Game Project

ESP32 Four Target Version

Prepared for the IoT Final Project and Intelligent Systems Integration

Group Members: _____

Course and Section: _____

Instructor: _____

Date: _____

Revision 1.0

DOCUMENT PURPOSE

This manual gives the group a complete path from purchasing parts to presenting a working four-target game. It covers the mechanical target, safe ESP32 wiring, firmware, calibration, testing, operation, troubleshooting, and preparation for later Intelligent Systems software. Build and test one target first, then repeat the proven circuit for the other three targets.

PROJECT SCOPE

- A player throws lightweight foam balls at four impact plates.
- The ESP32 lights one random target at a time for a limited target window.
- A correct hit scores 10 points; wrong hits and missed targets are recorded.
- A 30-second round shows time and score on an OLED display and uses a buzzer for feedback.
- Serial game records prepare the project for a future leaderboard, player profile, and performance analysis application.

IMPORTANT SAFETY LIMITS

- Use only soft foam balls. Do not use marbles, stones, metal balls, or hard plastic balls.
- Disconnect power before changing wiring or soldering.
- Never connect 5 V directly to an ESP32 GPIO pin. GPIO signals are 3.3 V logic.
- Cover solder joints, sharp fasteners, and exposed wire ends before public use.
- Use a regulated 5 V supply. Do not build or modify mains-voltage wiring.

TABLE OF CONTENTS

DOCUMENT PURPOSE	2
TABLE OF CONTENTS.....	3
1 PROJECT OVERVIEW	4
2 GAME RULES AND SETTINGS.....	5
3 COMPONENTS AND BUYING GUIDE.....	6
4 ELECTRICAL DESIGN	8
5 WIRING THE PROTOTYPE.....	10
6 BUILDING THE TARGET MECHANISM	13
7 MOVING FROM BREADBOARD TO PERFBOARD	15
8 SOFTWARE SETUP	16
9 COMPLETE ESP32 PROGRAM.....	17
10 UPLOAD AND FIRST RUN	22
11 CALIBRATION AND TESTING	23
12 TROUBLESHOOTING.....	24
13 ONE MONTH BUILD SCHEDULE.....	25
14 INTELLIGENT SYSTEMS READINESS.....	26
15 OPERATION AND MAINTENANCE.....	27
16 FINAL READINESS CHECKLIST.....	28

1 PROJECT OVERVIEW

Game Concept

I-Shoot Mo, I-Shoot Mo is a sensor-based carnival throwing game. Four target plates are mounted on a decorated backboard. When one target lights, the player throws a foam ball at that plate. The plate presses a protected microswitch, allowing the ESP32 to validate the hit, calculate the reaction time, update the score, and select another target.

Expected Demonstration

- The game completes a power-on light and buzzer test.
- The START button begins a three-second countdown and a 30-second round.
- Only the illuminated target earns points.
- The OLED shows the remaining time, active target, score, correct hits, wrong hits, and misses.
- The target time becomes shorter after every five correct hits, down to 650 milliseconds.
- The final screen shows the score and performance summary.

SYSTEM BLOCK DIAGRAM

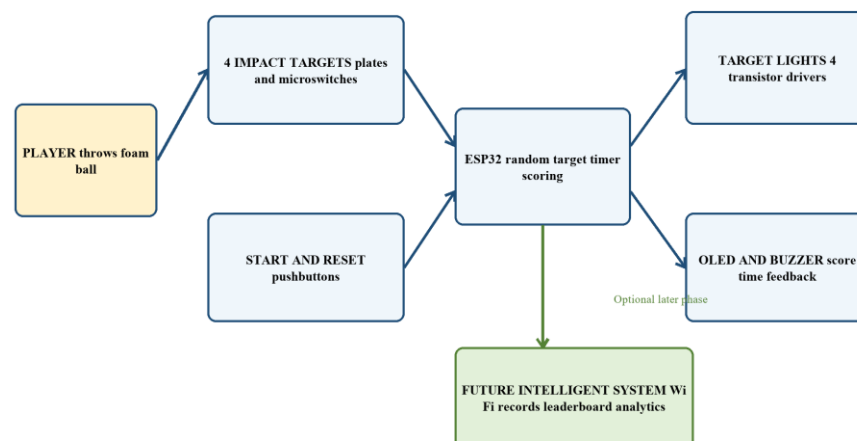


Figure 1. System block diagram for the complete project

Success Criteria

Test Area	Acceptance Check
Target detection	At least 19 of 20 firm test presses register on each target.
False triggers	No false hit occurs during a two-minute idle test and normal table vibration.

Test Area	Acceptance Check
Lighting	Only one target light is active during normal play.
Game timing	A round lasts 30 seconds within normal human-observable tolerance.
Public safety	Only foam balls are used, and all electronics and sharp parts are covered.
Data output	The Serial Monitor prints start, target, hit, wrong, miss, and end records.

2 GAME RULES AND SETTINGS

Default Rules

1. The operator retrieves the balls and confirms that all target plates move freely.
2. The player stands behind the marked throwing line and presses START.
3. After the three-second countdown, one target lights.
4. A correct hit adds 10 points and immediately activates the next target.
5. A wrong target records an error but does not subtract points.
6. A target that is not hit within its time window records a miss.
7. The round ends after 30 seconds. The OLED shows the final result.

Settings That Can Be Changed

Code Setting	Default	Meaning
ROUND_MS	30000	Total round time in milliseconds
START_WINDOW_MS	1500	Initial time allowed for one target
MIN_WINDOW_MS	650	Fastest target window
WINDOW_STEP_MS	100	Reduction after every five correct hits
DEBOUNCE_MS	45	Switch bounce filter
Score line	score += 10	Points given for a correct hit

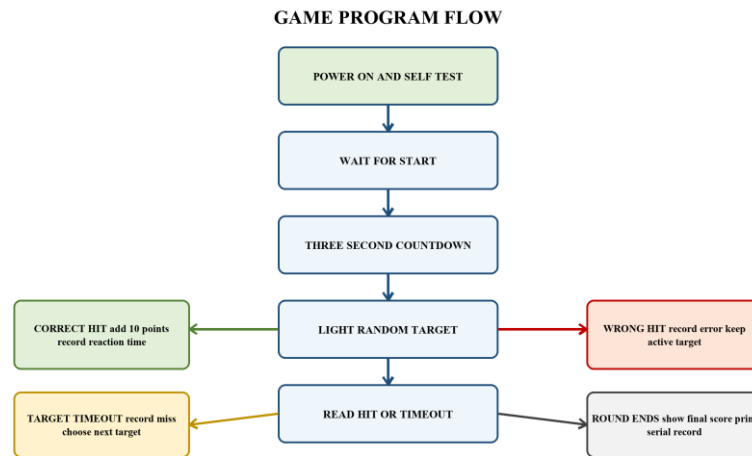


Figure 2. Program flow from power-on to final score

3 COMPONENTS AND BUYING GUIDE

The prices below are planning ranges in Philippine pesos, not fixed quotations. Online prices, shipping fees, seller ratings, and stock change frequently. Each link opens a Shopee Philippines search page so the group can compare current listings. Check the exact voltage, pin type, dimensions, and seller reviews before purchasing.

Component and Specification	Qty	Estimated Cost	Purpose	Shopee
ESP32 DevKit V1, ESP32-WROOM-32	1	PHP 250–400	Main controller	Open
0.96-inch SSD1306 OLED, I2C, 128×64	1	PHP 100–180	Score and timer display	Open
KW11 or similar lever microswitch	6	PHP 60–150	Impact detection plus spares	Open
5 mm high-brightness LEDs, assorted	12	PHP 30–100	Target indicators	Open
2N2222 NPN transistor	6	PHP 20–60	LED switching plus spares	Open
220 ohm resistor, 0.25 W	15	PHP 20–50	One per LED	Open
1 kohm resistor, 0.25 W	6	PHP 20–50	Transistor base limit	Open
10 kohm resistor, 0.25 W	6	PHP 20–50	Base pulldown	Open
Three-pin active buzzer module	1	PHP 25–70	Sound feedback	Open
Momentary pushbuttons for START and RESET	2	PHP 20–60	Player controls	Open
Full-size breadboard	1	PHP 80–150	Temporary prototype	Open
Male-to-male and male-to-female jumpers	1 set	PHP 80–150	Prototype connections	Open

Component and Specification	Qty	Estimated Cost	Purpose	Shopee
Large perfboard or universal PCB	1–2	PHP 50–170	Final soldered circuit	Open
Screw terminals, headers, and connectors	1 set	PHP 80–180	Removable wiring	Open
Regulated 5 V 2 A adapter	1	PHP 150–300	Power source	Open
ESP32 USB data cable	1	PHP 50–100	Programming and power	Open
470 uF capacitor and 100 nF capacitors	1 set	PHP 30–80	Power filtering	Open
22 AWG stranded hookup wire	5 m	PHP 80–150	Long target wiring	Open
Soft foam balls, 5–7 cm diameter	6–10	PHP 100–250	Safe throwing pieces	Open
Thin plywood or plastic for impact plates	1 sheet	PHP 100–250	Moving target plates	Open
Plywood for backboard and base	1 set	PHP 200–450	Main structure	Open
Cardboard or foam board for decoration	1 set	PHP 100–250	Carnival finish	Open
Foam pads, hinges or nylon straps	1 set	PHP 80–200	Plate return mechanism	Open
Screws, glue, tape, paint, and cable ties	1 set	PHP 250–500	Assembly and finish	Open

Estimated Budget

The listed planning ranges total PHP 1,995 to PHP 4,350 before shipping. A practical group budget is PHP 3,200, including small replacement parts. The total can be lower when the group already owns an ESP32, breadboard, jumper wires, tools, paint, plywood scraps, or a suitable 5 V adapter.

Tools Not Included in the Budget

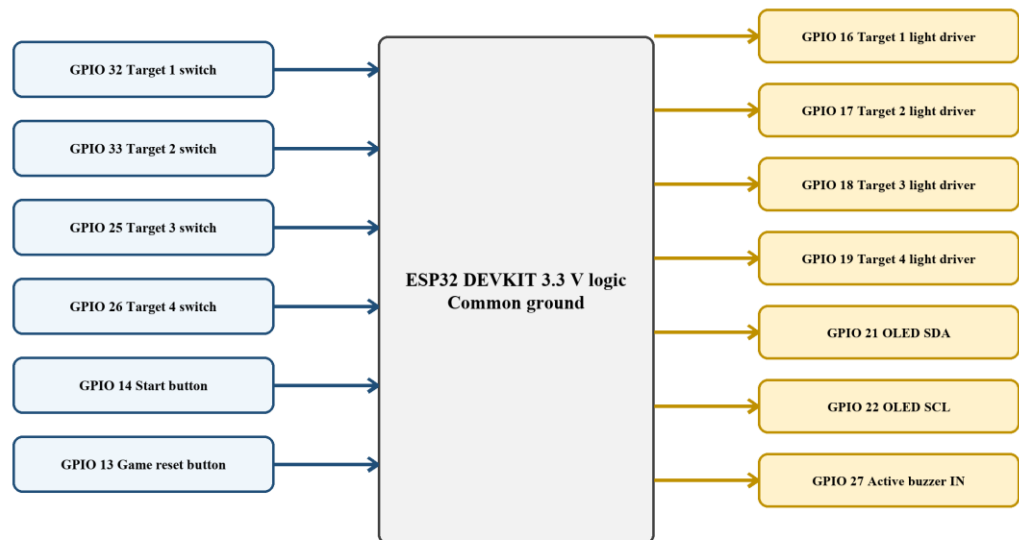
- Soldering iron, solder, stand, and a damp cleaning sponge
- Digital multimeter
- Wire stripper, side cutter, long-nose pliers, and screwdriver set
- Hand drill, small drill bits, craft knife, ruler, pencil, and cutting mat
- Hot-glue gun, clamps, sandpaper, and paintbrushes

4 ELECTRICAL DESIGN

Pin Assignment

Function	ESP32 Pin	Mode	Connection
Target 1 switch	GPIO 32	INPUT_PULLUP	NO switch terminal to GPIO; COM to GND
Target 2 switch	GPIO 33	INPUT_PULLUP	NO switch terminal to GPIO; COM to GND
Target 3 switch	GPIO 25	INPUT_PULLUP	NO switch terminal to GPIO; COM to GND
Target 4 switch	GPIO 26	INPUT_PULLUP	NO switch terminal to GPIO; COM to GND
Target 1 light	GPIO 16	OUTPUT	Through 1 kohm resistor to 2N2222 base
Target 2 light	GPIO 17	OUTPUT	Through 1 kohm resistor to 2N2222 base
Target 3 light	GPIO 18	OUTPUT	Through 1 kohm resistor to 2N2222 base
Target 4 light	GPIO 19	OUTPUT	Through 1 kohm resistor to 2N2222 base
START button	GPIO 14	INPUT_PULLUP	Button to GND
GAME RESET button	GPIO 13	INPUT_PULLUP	Button to GND
OLED SDA	GPIO 21	I2C data	OLED powered from 3.3 V
OLED SCL	GPIO 22	I2C clock	OLED powered from 3.3 V
Buzzer module IN	GPIO 27	OUTPUT	Module VCC per its label; common GND

ESP32 PIN ASSIGNMENT



Avoid 5 V on every GPIO pin. Switch inputs connect to GND and use INPUT_PULLUP.

Figure 3. ESP32 pin assignment used by the supplied program

Power Plan

- Use one regulated 5 V 2 A adapter for the completed game.
- Connect 5 V to the ESP32 VIN or 5V pin only when the board is not also receiving power from another source. During programming, the safest method is to power the ESP32 from USB and keep the external lighting supply disconnected.
- Power the OLED from the ESP32 3.3 V pin.
- Power the LED anodes from the 5 V rail through individual 220 ohm resistors.
- Connect the ESP32 GND, 5 V supply negative, transistor emitters, switch returns, OLED GND, and buzzer GND to one common ground.
- Place a 470 uF capacitor across 5 V and GND near the LED drivers. Observe capacitor polarity.

5 WIRING THE PROTOTYPE

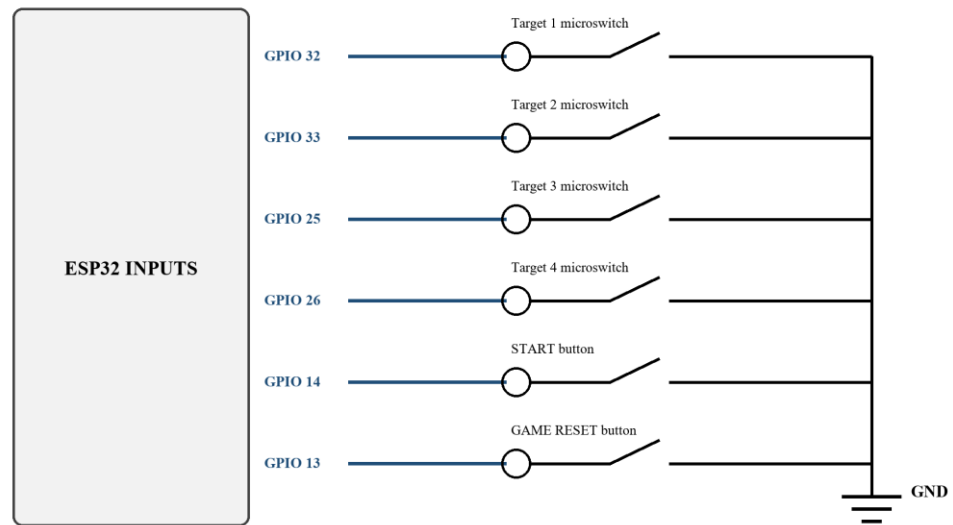
Build Order

1. Keep all power disconnected. Place the ESP32 across the breadboard center gap.
2. Connect the breadboard ground rail to an ESP32 GND pin.
3. Wire the OLED to 3.3 V, GND, GPIO 21, and GPIO 22.
4. Wire START between GPIO 14 and GND. Wire GAME RESET between GPIO 13 and GND.
5. Wire one microswitch first: COM to GND and NO to GPIO 32. Leave the NC terminal unused.
6. Build one transistor LED driver using GPIO 16. Confirm the transistor pin order from the exact seller datasheet because 2N2222 packages do not all use the same physical arrangement.
7. Upload the program and test Target 1. Repeat the proven switch and light circuits for Targets 2 through 4.
8. Add the active buzzer module: IN to GPIO 27, GND to common GND, and VCC according to the module marking.
9. Test the complete game on the breadboard before cutting the enclosure or soldering the perfboard.

Target Switch Wiring

Each switch input uses the ESP32 internal pull-up resistor. An unpressed switch reads HIGH. Pressing the plate closes the NO contact to ground, so the input reads LOW. This arrangement needs no external input resistor and reduces loose-wire false readings.

INPUT WIRING FOR TARGETS AND CONTROL BUTTONS



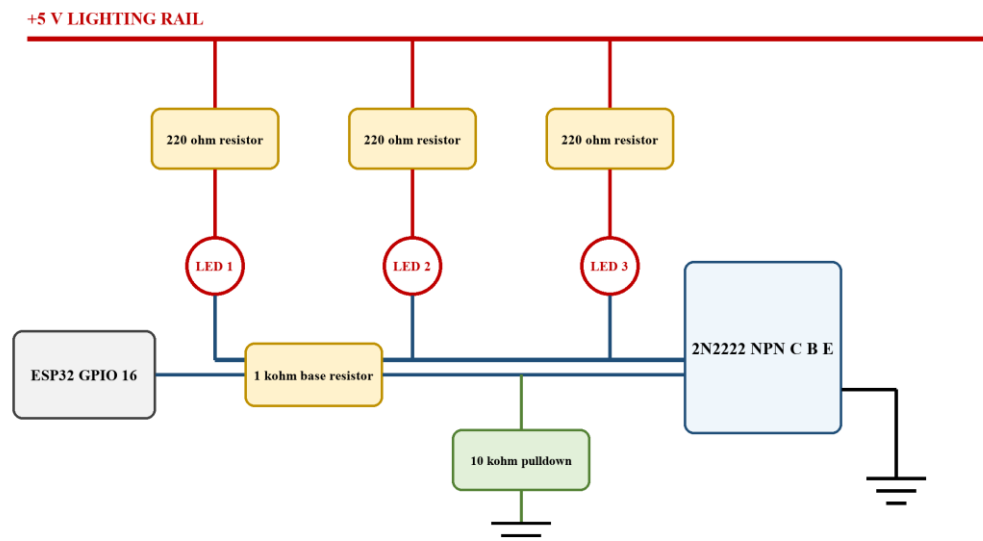
Use the NO and COM terminals. A press connects the GPIO input to GND.

Figure 4. Microswitch and control-button input wiring

Target Light Driver

Each target uses three LEDs. Give every LED its own 220 ohm resistor. The three LED branches share one 2N2222 transistor, which acts as a low-side switch. The ESP32 drives the transistor base through a 1 kohm resistor, and a 10 kohm pulldown keeps the light off while the controller starts.

ONE TARGET LIGHT DRIVER REPEAT FOUR TIMES

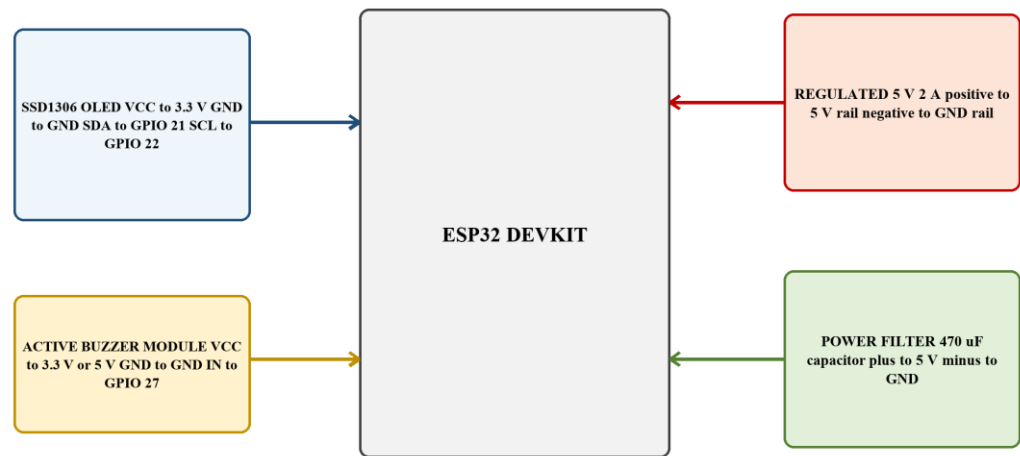


The transistor keeps LED current away from the ESP32 GPIO. All grounds must be connected.

Figure 5. One target light driver repeated for Targets 1 through 4

OLED Buzzer and Supply

OLED BUZZER AND POWER WIRING



Do not connect the 5 V rail directly to a GPIO pin. Connect every module ground to the same GND rail.

Figure 6. OLED, buzzer, power supply, and common-ground connections

Prepower Inspection

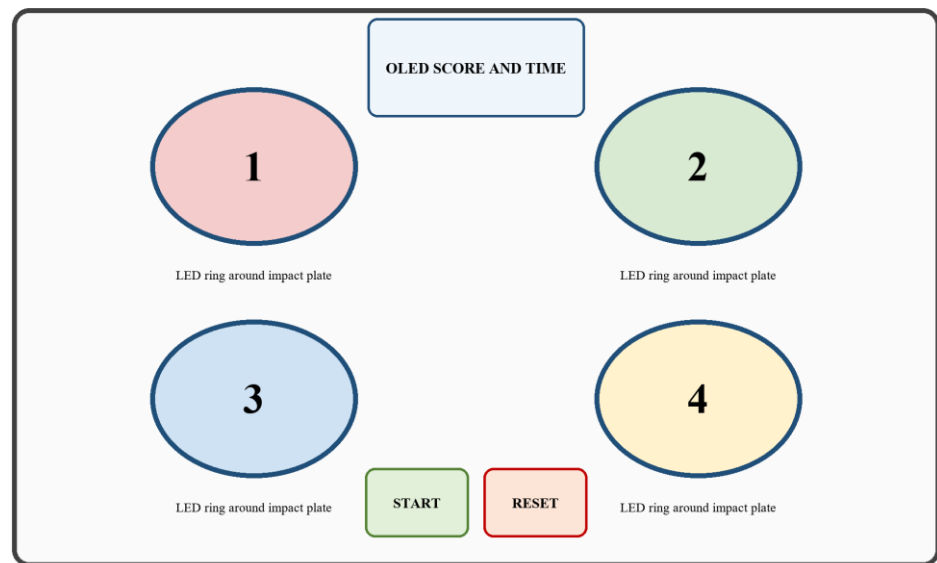
- No 5 V wire reaches GPIO 13, 14, 16, 17, 18, 19, 21, 22, 25, 26, 27, 32, or 33.
- Each LED has its own 220 ohm series resistor.
- Every transistor emitter reaches GND, and its collector reaches the LED cathode bus.
- The electrolytic capacitor negative stripe reaches GND.
- The multimeter shows no direct short between 5 V and GND.
- All long wires have labels on both ends.

6 BUILDING THE TARGET MECHANISM

Recommended Dimensions

- Backboard: about 120 cm wide by 90 cm high
- Impact plate: 15 to 20 cm across
- Space between target centers: at least 35 cm
- Plate movement before switch activation: about 3 to 8 mm
- Throwing line: begin testing at 1.5 m and adjust for the audience

SUGGESTED FRONT PANEL LAYOUT



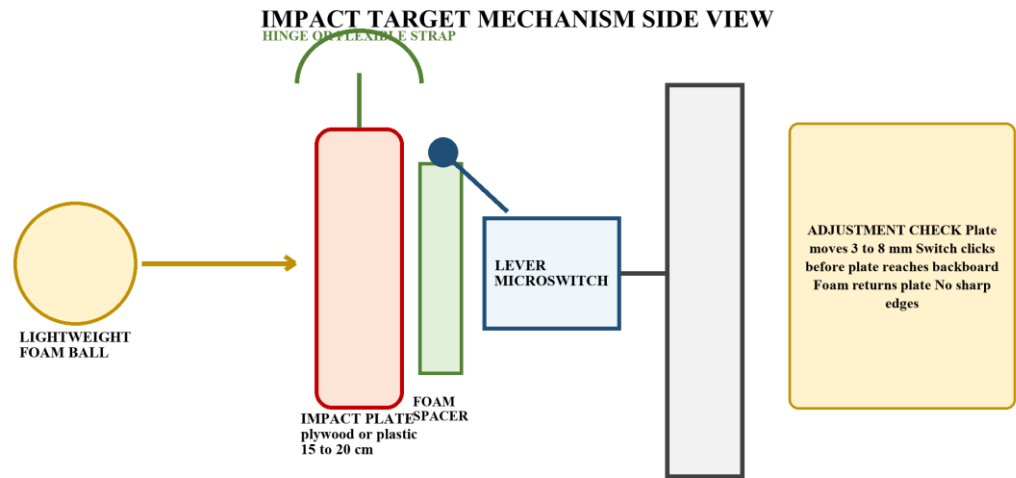
Recommended board size about 120 cm wide by 90 cm high. Adjust to available materials.

Figure 7. Suggested four-target front panel and control layout

One Target Assembly

1. Cut a round or rounded impact plate from 3 to 6 mm plywood or durable plastic. Sand every edge.
2. Attach the top of the plate to the backboard with a small hinge, nylon strap, or flexible plastic strip.
3. Place soft foam spacers behind the left and right edges. The plate should return to its original position after a hit.
4. Mount the lever microswitch behind the plate. Adjust its position so a firm plate movement causes one clear click.
5. Add a physical stop so a hard throw cannot crush the microswitch body.
6. Install the three LEDs around the plate or behind a translucent ring. Keep LED legs and wires behind the front surface.

7. Press the plate at the center and near each edge. Reposition the switch or add a thin pressure pad until all normal hits register.



Mount LEDs around the plate or behind a translucent ring. Do not expose the switch to direct ball impact.

Figure 8. Protected target mechanism with adjustable switch position

Enclosure and Finish

- Mount the electronics in a rear box with ventilation and a removable service cover.
- Secure the ESP32 and perfboard with standoffs rather than loose tape.
- Route signal wires separately from moving target parts.
- Add strain relief where the power cable enters the enclosure.
- Use carnival colors and large target numbers, but keep the OLED and START button easy to see.
- Place a ball-return tray or fabric catch area below the targets when space permits.

7 MOVING FROM BREADBOARD TO PERFBOARD

Recommended Board Sections

- Section A contains screw terminals for 5 V and GND.
- Section B contains four 2N2222 transistors and their 1 kohm and 10 kohm resistors.
- Section C contains removable connectors for the target LEDs and switches.
- Section D contains OLED, buzzer, START, RESET, and ESP32 header connections.

Soldering Procedure

1. Photograph the working breadboard and mark every wire by function.
2. Draw the perfboard plan on paper. Confirm component orientation and connector labels.
3. Solder the power rails first. Test continuity and confirm that 5 V is not shorted to GND.
4. Solder one transistor driver and test it with its target before adding the remaining three.
5. Use sockets or female headers for the ESP32 so it can be removed.
6. Use screw terminals or locking connectors for wires that travel to the moving targets.
7. Trim lead ends, clean the board, inspect for solder bridges, and cover the bottom with insulated standoffs.

Wire Labels

Label	Meaning
T1-SW	Target 1 switch to GPIO 32
T2-SW	Target 2 switch to GPIO 33
T3-SW	Target 3 switch to GPIO 25
T4-SW	Target 4 switch to GPIO 26
T1-L	Target 1 light from Driver 1
T2-L	Target 2 light from Driver 2
T3-L	Target 3 light from Driver 3
T4-L	Target 4 light from Driver 4

8 SOFTWARE SETUP

Arduino IDE Preparation

1. Install Arduino IDE 2.x on the group laptop.
2. Open File, Preferences and add the official Espressif ESP32 board-manager URL if ESP32 support is not installed.
3. Open Boards Manager, search for esp32 by Espressif Systems, and install the package.
4. Open Library Manager and install Adafruit GFX Library and Adafruit SSD1306.
5. Connect the ESP32 with a USB data cable. Select the detected port and the matching ESP32 Dev Module board.
6. Create a sketch named I_Shoot_Mo and replace its contents with the complete program in Section 9.

Library and Board References

- [Arduino ESP32 documentation](#)
- [ESP32 GPIO reference](#)
- [ESP32 I2C reference](#)
- [Adafruit SSD1306 library](#)
- [Adafruit GFX library](#)

Before Uploading

- Disconnect the external 5 V lighting supply while the ESP32 is connected to the computer by USB.
- Confirm that the OLED address is 0x3C. If the display stays blank, run an I2C scanner and check for 0x3D.
- Confirm that the active buzzer turns on with a HIGH input. If the module is active-low, reverse the two HIGH and LOW states inside the beep function.
- Confirm the 2N2222 emitter, base, and collector order for the exact package purchased.

9 COMPLETE ESP32 PROGRAM

Copy the following code into the Arduino IDE. The program is complete for the four-target wiring in this manual. It does not require Wi-Fi for the first showcase. It prints structured records at 115200 baud so the later Intelligent Systems application can capture game data.

```
#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// I-Shoot Mo, I-Shoot Mo
// Four-target ESP32 carnival game

constexpr uint8_t SCREEN_WIDTH = 128;
constexpr uint8_t SCREEN_HEIGHT = 64;
constexpr int8_t OLED_RESET = -1;
constexpr uint8_t OLED_ADDRESS = 0x3C;

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

constexpr uint8_t NUM_TARGETS = 4;
constexpr uint8_t SWITCH_PINS[NUM_TARGETS] = {32, 33, 25, 26};
constexpr uint8_t LIGHT_PINS[NUM_TARGETS] = {16, 17, 18, 19};
constexpr uint8_t START_PIN = 14;
constexpr uint8_t RESET_PIN = 13;
constexpr uint8_t BUZZER_PIN = 27;
constexpr uint8_t SDA_PIN = 21;
constexpr uint8_t SCL_PIN = 22;

constexpr unsigned long ROUND_MS = 30000;
constexpr unsigned long START_WINDOW_MS = 1500;
constexpr unsigned long MIN_WINDOW_MS = 650;
constexpr unsigned long WINDOW_STEP_MS = 100;
constexpr unsigned long DISPLAY_REFRESH_MS = 100;
constexpr unsigned long DEBOUNCE_MS = 45;

enum GameState { IDLE, PLAYING, RESULTS };
GameState state = IDLE;

struct DebouncedInput {
    uint8_t pin;
    bool rawState;
    bool stableState;
    unsigned long changedAt;
};

constexpr uint8_t NUM_INPUTS = NUM_TARGETS + 2;
DebouncedInput inputs[NUM_INPUTS];

int8_t activeTarget = -1;
int8_t previousTarget = -1;
unsigned long roundStartedAt = 0;
unsigned long targetStartedAt = 0;
unsigned long targetWindowMs = START_WINDOW_MS;
unsigned long lastDisplayAt = 0;

uint16_t score = 0;
uint16_t correctHits = 0;
uint16_t wrongHits = 0;
uint16_t misses = 0;
unsigned long totalReactionMs = 0;
unsigned long bestReactionMs = 0;

void allLightsOff() {
    for (uint8_t i = 0; i < NUM_TARGETS; i++) {
        digitalWrite(LIGHT_PINS[i], LOW);
    }
}
```

```

void beep(uint16_t durationMs, uint8_t repeats = 1) {
    for (uint8_t i = 0; i < repeats; i++) {
        digitalWrite(BUZZER_PIN, HIGH);
        delay(durationMs);
        digitalWrite(BUZZER_PIN, LOW);
        if (i + 1 < repeats) delay(60);
    }
}

bool pressedEvent(uint8_t inputIndex) {
    DebouncedInput &input = inputs[inputIndex];
    bool raw = digitalRead(input.pin);
    unsigned long now = millis();

    if (raw != input.rawState) {
        input.rawState = raw;
        input.changedAt = now;
    }

    if ((now - input.changedAt >= DEBOUNCE_MS) &&
        (input.stableState != input.rawState)) {
        input.stableState = input.rawState;
        return input.stableState == LOW;
    }
    return false;
}

void printCentered(const String &text, uint8_t size, int16_t y) {
    int16_t x1, y1;
    uint16_t width, height;
    display.setTextSize(size);
    display.getTextBounds(text, 0, y, &x1, &y1, &width, &height);
    display.setCursor((SCREEN_WIDTH - width) / 2, y);
    display.print(text);
}

void showIdle() {
    display.clearDisplay();
    display.setTextColor(SSD1306_WHITE);
    printCentered("I-SHOOT MO", 2, 5);
    printCentered("I-SHOOT MO", 2, 24);
    printCentered("Press START", 1, 51);
    display.display();
}

void showCountdown(uint8_t number) {
    display.clearDisplay();
    display.setTextColor(SSD1306_WHITE);
    printCentered("GET READY", 2, 7);
    printCentered(String(number), 4, 30);
    display.display();
}

void showGame(unsigned long now) {
    unsigned long elapsed = now - roundStartedAt;
    unsigned long remaining = elapsed >= ROUND_MS ? 0 : ROUND_MS - elapsed;

    display.clearDisplay();
    display.setTextColor(SSD1306_WHITE);
    display.setTextSize(1);
    display.setCursor(0, 0);
    display.print("Time: ");
    display.print((remaining + 999) / 1000);
    display.print(" s");
    display.setCursor(72, 0);
    display.print("Score:");
    display.print(score);
    display.setTextSize(2);
    display.setCursor(0, 19);
    display.print("TARGET ");
    display.print(activeTarget + 1);
    display.setTextSize(1);
    display.setCursor(0, 46);

```

```

    display.print("Hit:"); display.print(correctHits);
    display.print(" Wrong:"); display.print(wrongHits);
    display.setCursor(0, 57);
    display.print("Miss:"); display.print(misses);
    display.print(" Win:"); display.print(targetWindowMs);
    display.display();
}

void showResults() {
    unsigned long average = correctHits ? totalReactionMs / correctHits : 0;
    display.clearDisplay();
    display.setTextColor(SSD1306_WHITE);
    display.setTextSize(1);
    display.setCursor(0, 0); display.print("ROUND COMPLETE");
    display.setTextSize(2);
    display.setCursor(0, 14); display.print("SCORE "); display.print(score);
    display.setTextSize(1);
    display.setCursor(0, 37); display.print("Hit:"); display.print(correctHits);
    display.print(" Wrong:"); display.print(wrongHits);
    display.setCursor(0, 48); display.print("Miss:"); display.print(misses);
    display.print(" Avg:"); display.print(average);
    display.setCursor(0, 58); display.print("START to play again");
    display.display();
}

void activateNextTarget() {
    allLightsOff();
    int8_t next;
    do {
        next = random(NUM_TARGETS);
    } while (NUM_TARGETS > 1 && next == previousTarget);

    previousTarget = next;
    activeTarget = next;
    targetStartedAt = millis();
    digitalWrite(LIGHT_PINS[activeTarget], HIGH);

    Serial.print("TARGET,");
    Serial.print(targetStartedAt - roundStartedAt);
    Serial.print(",");
    Serial.println(activeTarget + 1);
}

void startRound() {
    allLightsOff();
    score = 0;
    correctHits = 0;
    wrongHits = 0;
    misses = 0;
    totalReactionMs = 0;
    bestReactionMs = 0;
    targetWindowMs = START_WINDOW_MS;
    previousTarget = -1;

    for (int8_t n = 3; n >= 1; n--) {
        showCountdown(n);
        beep(80);
        delay(920);
    }

    roundStartedAt = millis();
    state = PLAYING;
    Serial.println("SESSION_START,30000");
    activateNextTarget();
}

void endRound() {
    allLightsOff();
    state = RESULTS;
    unsigned long average = correctHits ? totalReactionMs / correctHits : 0;
    beep(120, 3);
    showResults();

    Serial.print("SESSION_END,");

```

```

    Serial.print(score); Serial.print(",");
    Serial.print(correctHits); Serial.print(",");
    Serial.print(wrongHits); Serial.print(",");
    Serial.print(misses); Serial.print(",");
    Serial.print(average); Serial.print(",");
    Serial.println(bestReactionMs);
}

void resetToIdle() {
    allLightsOff();
    digitalWrite(BUZZER_PIN, LOW);
    activeTarget = -1;
    state = IDLE;
    showIdle();
    Serial.println("GAME_RESET");
}

void processPlaying(unsigned long now) {
    if (now - roundStartedAt >= ROUND_MS) {
        endRound();
        return;
    }

    for (uint8_t i = 0; i < NUM_TARGETS; i++) {
        if (!pressedEvent(i)) continue;

        if (i == activeTarget) {
            unsigned long reaction = now - targetStartedAt;
            correctHits++;
            score += 10;
            totalReactionMs += reaction;
            if (bestReactionMs == 0 || reaction < bestReactionMs) {
                bestReactionMs = reaction;
            }

            Serial.print("HIT,");
            Serial.print(now - roundStartedAt); Serial.print(",");
            Serial.print(i + 1); Serial.print(",");
            Serial.print(reaction); Serial.print(",");
            Serial.println(score);

            beep(55);
            if (correctHits % 5 == 0 && targetWindowMs > MIN_WINDOW_MS) {
                if (targetWindowMs > MIN_WINDOW_MS + WINDOW_STEP_MS) {
                    targetWindowMs -= WINDOW_STEP_MS;
                } else {
                    targetWindowMs = MIN_WINDOW_MS;
                }
            }
            activateNextTarget();
        } else {
            wrongHits++;
            Serial.print("WRONG,");
            Serial.print(now - roundStartedAt); Serial.print(",");
            Serial.print(i + 1); Serial.print(",");
            Serial.print(activeTarget + 1); Serial.print(",");
            Serial.println(score);
            beep(35, 2);
        }
    }

    if (now - targetStartedAt >= targetWindowMs) {
        misses++;
        Serial.print("MISS,");
        Serial.print(now - roundStartedAt); Serial.print(",");
        Serial.print(activeTarget + 1); Serial.print(",");
        Serial.println(score);
        activateNextTarget();
    }

    if (now - lastDisplayAt >= DISPLAY_REFRESH_MS) {
        lastDisplayAt = now;
        showGame(now);
    }
}

```

```

}

void setup() {
  Serial.begin(115200);

  for (uint8_t i = 0; i < NUM_TARGETS; i++) {
    pinMode(LIGHT_PINS[i], OUTPUT);
    digitalWrite(LIGHT_PINS[i], LOW);
  }
  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);

  const uint8_t pins[NUM_INPUTS] = {
    SWITCH_PINS[0], SWITCH_PINS[1], SWITCH_PINS[2], SWITCH_PINS[3],
    START_PIN, RESET_PIN
  };
  for (uint8_t i = 0; i < NUM_INPUTS; i++) {
    inputs[i].pin = pins[i];
    pinMode(inputs[i].pin, INPUT_PULLUP);
    inputs[i].rawState = digitalRead(inputs[i].pin);
    inputs[i].stableState = inputs[i].rawState;
    inputs[i].changedAt = millis();
  }

  Wire.begin(SDA_PIN, SCL_PIN);
  if (!display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDRESS)) {
    Serial.println("ERROR,OLED_NOT_FOUND");
    while (true) {
      digitalWrite(BUZZER_PIN, HIGH); delay(100);
      digitalWrite(BUZZER_PIN, LOW); delay(900);
    }
  }

  display.clearDisplay();
  display.display();
  randomSeed(analogRead(34) ^ micros());

  for (uint8_t i = 0; i < NUM_TARGETS; i++) {
    digitalWrite(LIGHT_PINS[i], HIGH); delay(120);
    digitalWrite(LIGHT_PINS[i], LOW);
  }
  beep(80, 2);
  showIdle();
  Serial.println("READY");
}

void loop() {
  unsigned long now = millis();

  if (pressedEvent(NUM_TARGETS + 1)) {
    resetToIdle();
    return;
  }

  if (state == IDLE || state == RESULTS) {
    if (pressedEvent(NUM_TARGETS)) startRound();
    return;
  }

  if (state == PLAYING) processPlaying(now);
}

```

10 UPLOAD AND FIRST RUN

1. Click Verify. Correct every compile error before connecting the finished hardware.
2. Click Upload. If the IDE cannot connect, hold BOOT while the upload begins, then release it.
3. Open Serial Monitor at 115200 baud. The display should show the title, and the Serial Monitor should print READY.
4. At startup, confirm that all four lights flash in order and the buzzer sounds twice.
5. Press each target manually and confirm that the corresponding input does not trigger another target.
6. Press START. Confirm the countdown, 30-second timer, random targets, score, errors, misses, and final screen.

Expected Serial Records

```
READY
SESSION_START,30000
TARGET,0,2
HIT,742,2,742,10
WRONG,1210,4,1,10
MISS,2850,1,10
SESSION_END,120,12,2,3,811,504
```

The SESSION_END fields are score, correct hits, wrong hits, misses, average reaction time in milliseconds, and best reaction time in milliseconds. These records form a simple interface for the later software project.

11 CALIBRATION AND TESTING

Target Calibration

1. Disconnect power and adjust the microswitch so the plate clicks after a small movement.
2. Press the center, top, bottom, left, and right areas of the plate ten times each.
3. Move the switch closer if light hits fail to register. Move it farther away or stiffen the foam if vibration causes false hits.
4. Add a hard stop behind the plate so the switch cannot carry the full impact force.
5. Repeat the test using the actual foam balls from the planned throwing distance.

Electrical Test Record

Test	Expected Result	Pass and Notes
Power off	No short between 5 V and GND	
Power on	5 V rail stays between about 4.75 V and 5.25 V	
OLED	Title, timer, score, and results are readable	
Target 1	19 of 20 test hits register	
Target 2	19 of 20 test hits register	
Target 3	19 of 20 test hits register	
Target 4	19 of 20 test hits register	
Idle vibration	No false hit for two minutes	
Full round	Correct score, errors, misses, and 30-second finish	
Ten rounds	No reset, loose wire, hot part, or unstable display	

Gameplay Calibration

- If beginners score fewer than three correct hits, increase START_WINDOW_MS to 1800 or 2000.
- If most players hit every target easily, move the throwing line back or reduce START_WINDOW_MS.
- Keep the minimum window at 650 milliseconds or higher until the mechanism is proven reliable.
- Use the same ball type and throwing distance during calibration and the public demonstration.

12 TROUBLESHOOTING

Problem	Likely Cause	Action
ESP32 does not power on	Wrong cable, weak supply, reversed power, or short	Disconnect everything. Test the USB data cable and ESP32 alone, then inspect the 5 V rail.
OLED is blank	Wrong address or SDA and SCL wiring	Check 3.3 V and GND, confirm GPIO 21 and 22, then scan for address 0x3C or 0x3D.
One target always reads pressed	COM wired to NC or signal shorted to GND	Move the wire to NO and inspect the cable for damage.
One target misses hits	Switch too far from plate or plate flexes unevenly	Move the switch closer and add a wider pressure pad behind the plate.
Multiple hits from one impact	Mechanical bounce exceeds the filter	Increase DEBOUNCE_MS from 45 to 60 or 80 and reduce plate vibration.
LED stays off	LED polarity, transistor pin order, or missing common ground	Test the LED branch, verify emitter-base-collector order, and confirm common GND.
LED is dim	Wrong resistor, weak supply, or poor connection	Confirm 220 ohm per LED, measure the 5 V rail, and inspect solder joints.
ESP32 resets when lights turn on	Voltage drop or poor grounding	Use the 5 V 2 A supply, add the 470 uF capacitor, shorten power wires, and improve ground connections.
Buzzer is always on	Active-low module or wrong signal pin	Confirm GPIO 27 and reverse the HIGH and LOW states in beep if required.
Upload fails	Wrong port, charge-only cable, or boot mode	Select the correct port, use a data cable, and hold BOOT while upload starts.

13 ONE MONTH BUILD SCHEDULE

Period	Focus	Main Work	Required Result
Week 1	Design and parts	Finalize dimensions; order parts; install Arduino IDE; test ESP32 and OLED; cut one cardboard target mock-up.	Approved dimensions, ordered parts, working controller and display
Week 2	One complete target	Build and calibrate one impact plate; prototype its switch and LED driver; test the game logic with one target.	One reliable target and confirmed circuit
Week 3	Full hardware	Build three remaining targets; wire the full breadboard; fabricate the backboard; prepare and solder the perfboard.	Four-target game operating on the enclosure
Week 4	Reliability and presentation	Run repeated games; fix weak wiring; paint and label the booth; rehearse operation; prepare spare parts and transport plan.	Demonstration-ready system and evidence of testing

Suggested Group Roles

- Project lead tracks the schedule, parts, test record, and instructor requirements.
- Firmware lead uploads the code, adjusts settings, and records the final version.
- Electronics lead builds the breadboard, perfboard, and labeled wiring.
- Mechanical lead builds the plates, backboard, enclosure, and ball-return area.
- Presentation and quality lead manages decoration, safety checks, game rules, and demonstration practice.

14 INTELLIGENT SYSTEMS READINESS

The first presentation can run entirely on the ESP32. The same hardware is ready for later software because it already produces structured event records. A laptop or web application can read the records through USB serial first. Wi-Fi transmission can be added after the physical game is stable.

Minimum Three Core Software Features

1. Player profiles and leaderboard. The application stores a player name or identifier, score, correct hits, errors, misses, average reaction time, and best reaction time.
2. Performance analysis. The application graphs accuracy and reaction time by round and identifies the target number with the most misses.
3. Adaptive difficulty. The application recommends an easier or harder target window based on the player's recent accuracy and reaction time.

Suggested Record Structure

Record	Fields	Meaning
SESSION_START	round duration	A new game begins
TARGET	elapsed time, target number	A target light turns on
HIT	elapsed time, target, reaction, score	The active target is pressed
WRONG	elapsed time, pressed target, active target, score	A nonactive target is pressed
MISS	elapsed time, active target, score	The target window expires
SESSION_END	score, correct, wrong, misses, average, best	The round finishes

Later Wi Fi Addition

- Keep the game loop local so the game remains playable if Wi-Fi fails.
- Send only completed events or a completed round summary to the application.
- Add a queue so records are not lost during a brief disconnection.
- Do not add Wi-Fi until the target mechanism, scoring, and power system pass the tests in Section 11.

15 OPERATION AND MAINTENANCE

Before Opening the Booth

- Inspect every target plate, screw, wire, and cover.
- Confirm that only soft foam balls are available.
- Run one complete test round and check the score, OLED, buzzer, and Serial Monitor.
- Mark the throwing line and keep spectators outside the ball path.
- Place spare foam balls, one microswitch, one transistor, resistors, tape, cable ties, and a USB cable near the operator.

Operator Procedure

1. Reset the game and explain the 30-second rule.
2. Give the player the approved foam balls and keep the next player behind the line.
3. Watch for unsafe throws or a loose target. Stop the round if needed.
4. Read the final score and retrieve the balls only after the round ends.
5. Inspect the plate if a hit does not register or a target remains pressed.

After Each Session

- Turn off the 5 V supply before opening the rear cover.
- Tighten loose connectors and remove damaged balls.
- Back up recorded scores if the future application is in use.
- Transport the ESP32 and perfboard in a padded, dry enclosure.

16 FINAL READINESS CHECKLIST

Done	Readiness Item	Notes
☐	ESP32 firmware uploads without errors.	
☐	All four switch inputs register the correct target.	
☐	All four target lights work and only one is active during play.	
☐	OLED shows countdown, time, target, score, and results.	
☐	Buzzer provides start, hit, and end feedback.	
☐	A full 30-second round ends correctly.	
☐	Target plates return after every hit and protect the switches.	
☐	No sharp edge, bare conductor, loose screw, or exposed circuit faces the player.	
☐	Only foam balls are packed for the demonstration.	
☐	The power adapter and USB cable are labeled and strain-relieved.	
☐	Serial Monitor records a complete round.	
☐	The group has spare parts, tools, printed rules, and assigned operator roles.	
☐	The game passes ten consecutive practice rounds without a reset or loose connection.	

Final Build Sequence

Purchase the parts, test the ESP32 and OLED, build one target, prove its switch and light circuit, repeat the circuit three times, assemble the backboard, transfer the tested wiring to perfboard, upload the final program, calibrate every plate, complete ten reliable practice rounds, and then finish the carnival decoration. This sequence keeps failures small and easy to locate.

Technical Notes

- The manual assumes a common ESP32 DevKit V1 board. Verify printed pin labels before wiring because board layouts vary.
- The LED circuit assumes red, yellow, or green 5 mm LEDs with one 220 ohm resistor per LED. If different lighting is used, recalculate current and use the correct driver. Keep the named connections even if final wire colors differ.